databricks

LLMOps Concepts

LECTURE

MLOps Primer

In this lecture MLOps Primer, we begin with conventional ML, explore the importance of MLOps, multi-environment semantics, environment separation, deployment patterns, recommended deploy-code architectures for MLOps, and modern LLMOps architecture on a unified platform.

Let us start from conventional ML

What is MLOps

MLOps is the set of **processes and automation**

for **managing data, code and models**

to **improve performance, stability and long-term efficiency** of ML systems

MLOps = DataOps + DevOps + ModelOps

Before diving into large language model operations, it's helpful to quickly revisit what machine learning operations, or MLOps, actually involve. MLOps is a set of processes and automated tools designed to manage data, code, and models, ensuring that the model continues to perform as intended over time. A typical machine learning system consists of three main components: the data or features, the models used for training and prediction, and the code that orchestrates interactions between the two.

For each pillar, there are rigorous best practices to ensure quality. The concept of MLOps brings these practices together into one unified workflow, with the goal that everything operates smoothly and in sync. This idea builds on established software development methods like DevOps, combined with specialized tools for experiment tracking, model management, and data operations to create a thorough machine learning lifecycle.

Why does MLOps matter?

Success depends on quality data and operations practices

- Defining an effective strategy
  - ML systems built on **quality data**
  - **Streamlining** process of taking solutions to **production**
  - Operationalizing **cost & performance monitoring**
- So what?
  - Accelerated time to realizing business value
  - Reduction in manual oversight

Real-world Example

Databricks customer CareSource accelerated their model's development and deployment, resulting in a **self-service MLOps solution for data scientists** that reduced ML project time from 8 weeks to 3-4 weeks.

The CareSource team can extend this approach to other machine learning projects, realizing this benefit broadly.

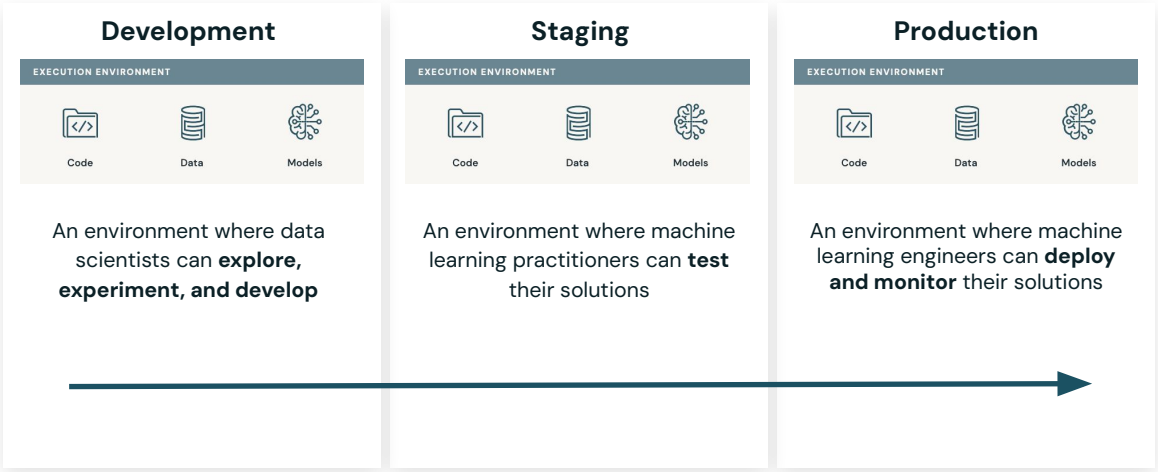
Learn more about the work here.

Why is this important? Well, the success of machine learning systems really hinges on following best practices in their operations. That means ensuring data quality, streamlining every stage of the process, and getting models into production as efficiently as possible. It's also crucial to monitor costs and performance so you can speed up time to market and achieve business value quickly.

If things aren't working as expected, having strong operational practices helps you catch problems early, reducing the need for intensive manual oversight or extra engineering work to resolve infrastructure issues.

Multi-environment Semantics

Defining Development, Staging, and Production environments



In traditional software development, you're always working across multiple environments. You start with a development environment, which is used for exploring ideas, experimenting, and building out your code. Sometimes there's also a staging environment, where you run functional tests and perform continuous integration or regression testing.

Ultimately, though, whatever you build is meant to end up in the production environment. That's where the system is fully deployed and monitored, powering your most important business applications.

Environment Separation

How many Databricks workspaces do we need?

Model Code

Model Artifact

Direct Separation

- Completed separate Databricks workspaces for each environment
- Simpler environments
- Scales well to multiple projects

Indirect Separation

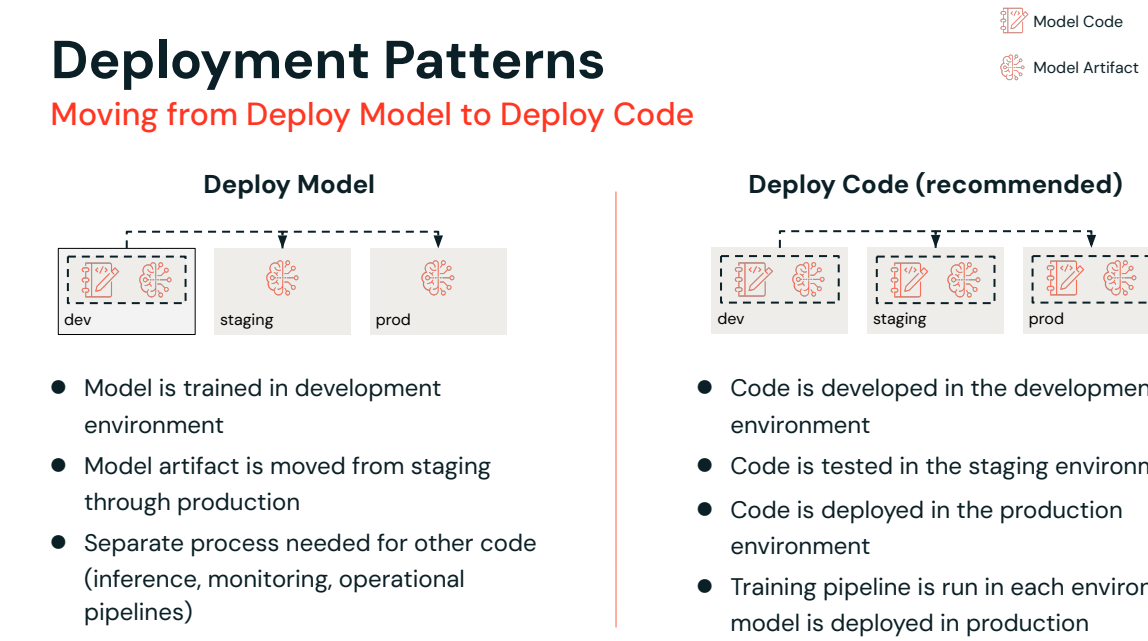
- One Databricks workspace with enforced separation
- Simpler overall infrastructure requiring less permission required
- Complex individual environment
- Doesn't scale well to multiple projects

In Databricks, environments can be separated using workspaces—for example, having one for development, one for staging, and one for production. This makes it easier to manage projects, reduce risk, and control access to sensitive or production data, with each workspace serving a specific purpose. Development environments offer flexibility for experimentation, staging is for automated workflow and code verification, and production handles critical applications and monitoring.

Alternatively, you can use a single workspace with strict access controls through Unity Catalog ACLs. This approach is simpler for quick setup and central management, but it requires careful stewardship to maintain permissions, data labeling, and resource limits. While indirect separation simplifies some aspects, it can become harder to scale with many models and projects due to API and hardware constraints. Ultimately, the best setup depends on specific requirements and organizational practices.

Deployment Patterns

Moving from Deploy Model to Deploy Code



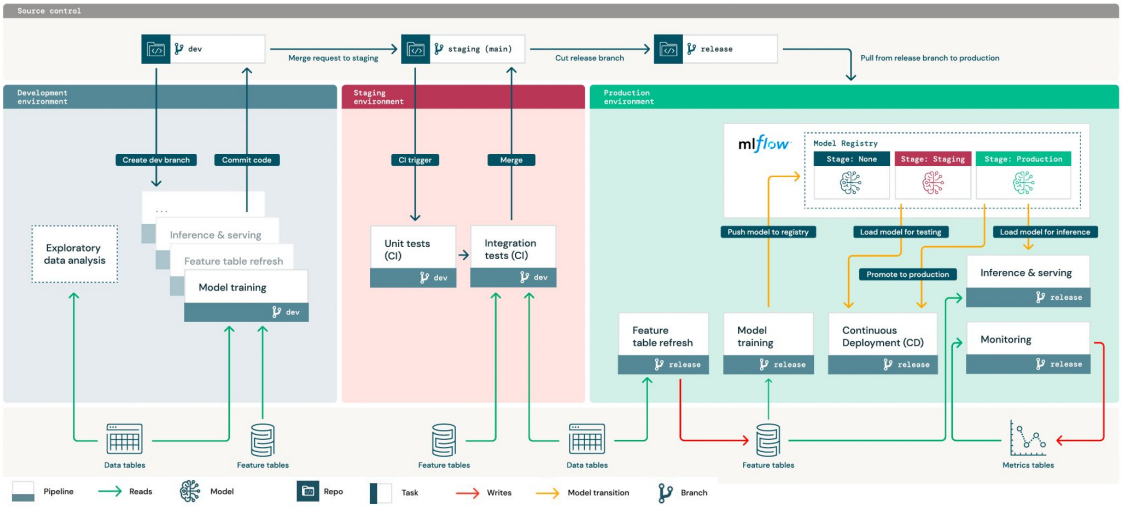
- Model is trained in development environment
- Model artifact is moved from staging through production
- Separate process needed for other code (inference, monitoring, operational pipelines)

- Code is developed in the development environment
- Code is tested in the staging environment
- Code is deployed in the production environment
- Training pipeline is run in each environment, model is deployed in production

In the machine learning and generative AI world, deployment can follow two main approaches: deploying code or deploying models. The traditional software route focuses on promoting code through different environments, where code changes determine what happens. In contrast, the deploy model approach allows you to train models in a development environment and then push those models to staging or production, often using automated jobs to handle validation and versioning.

With deploy model, the training happens in development, but the inference, monitoring, and production-side adjustments are addressed separately at the production level. Meanwhile, the deploy code strategy promotes and tests the entire pipeline across all stages—development, staging, and production—ensuring that both code and models are handled together throughout the workflow.

Recommended “deploy-code” architecture (MLOps)

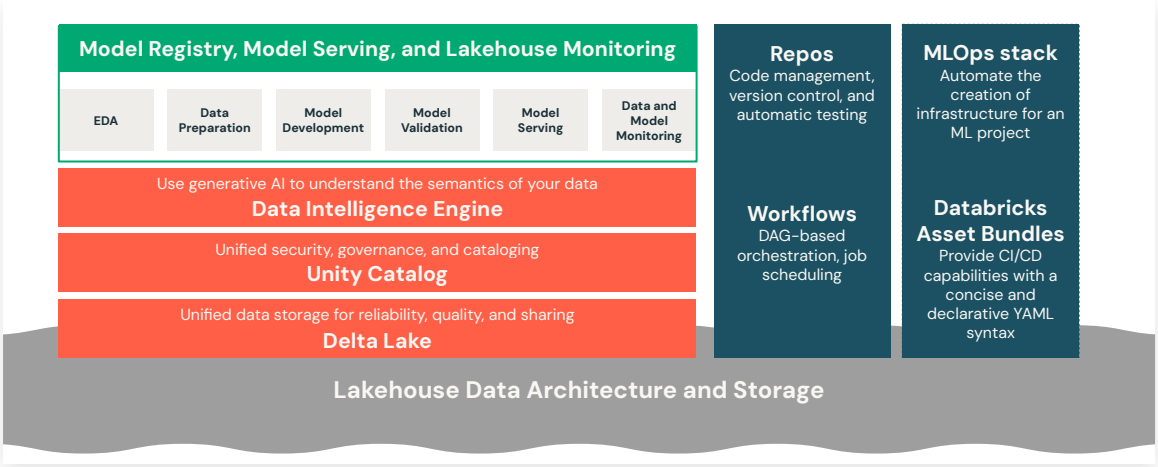


To illustrate how the workflow comes together, imagine starting in the development environment, where data scientists carry out exploratory analysis, build features, write inference and serving code, and train models using available data. Once changes are made, code gets committed to the dev branch, which triggers a pull request and launches unit and integration tests through CI/CD tools like Git Actions or Azure DevOps.

After tests pass, the code is merged into the staging branch to prepare a release, which becomes the reference for deploying in production. In the production environment, models go through feature updates, retraining, promotion, and testing before being deployed for API or batch jobs. Throughout this cycle, monitoring metrics are written back to the lakehouse, with all environments sharing a single source of truth—reinforcing a data-centric approach.

A Single Platform for Modern MLOps

Combining DataOps, DevOps, and ModelOps solutions

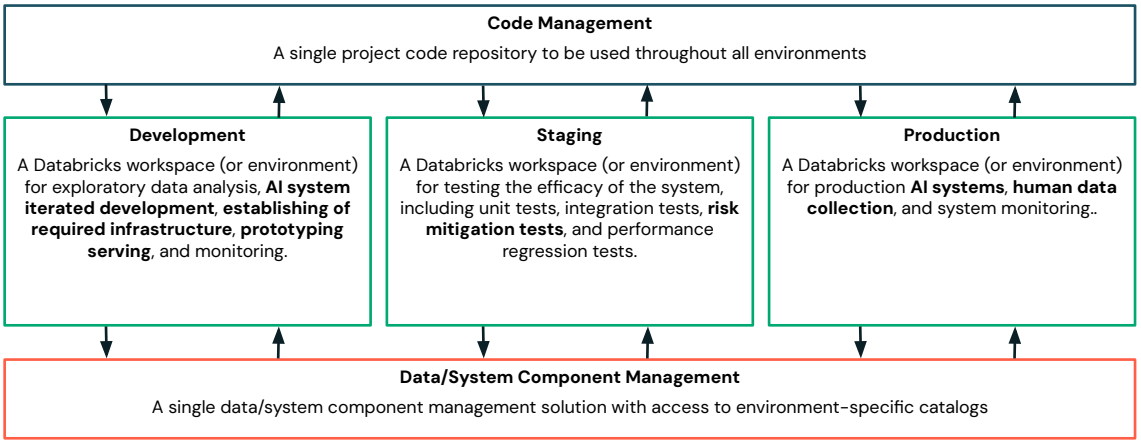


This unified platform approach is central to Databricks Intelligence Platform, allowing DataOps, DevOps, and ModelOps to operate together seamlessly—without the need for separate tools or stacks. In this course, the focus has mainly been on ModelOps, such as creating experiments, tracking progress, promoting models through the registry, serving them, and monitoring their performance. Unity Catalog plays a big part in DataOps too, by letting you track lineage and see how indexes and tables connect to your models and chains.

DevOps, although covered in other modules, completes the picture by handling integration with Git, version control, workflow scheduling, and infrastructure management for machine learning projects. Databricks provides MLOps tools and asset bundles that use a simple, declarative setup for CI/CD and orchestration. Overall, this cohesive platform means you can manage data, code, and models all in one place for efficient and scalable operations.

Recommended LLMOps Architecture

A high-level view of code, data, and GenAI environments



When it comes to LLM operations, start by developing iteratively in the development environment. This is where you prototype the model, get a sense of infrastructure needs, and learn how to best structure and parse messages for monitoring. In staging, extra risk mitigation steps are crucial—you'll add tests to check model quality and avoid harmful completions before moving forward.

In production, gathering and prioritizing human feedback becomes key, since evaluating unstructured text output is complex and hard to automate. Human insights help guide and improve future model iterations. Throughout, a data-centric approach ensures all environments leverage the same data management and code promotion systems for efficient and reliable operations.

Thank you for completing this lesson and continuing your journey to develop your skills with us.